CSO 정산 포털

v0.16.0

CSO(위탁영업) 업체가 제약사 정산 수수료를 조회하는 B2B 포털
기술 문서 통합본

Next.js TypeScript Supabase Tailwind CSS Netlify

2026-02-26 기준

영업관리팀 권대환 — qwer@ukp.co.kr

목차

1. CSO 정산 포털
2. 이관 매뉴얼
3. 온보딩 가이드
4. 배포 & 운영
5. 아키텍처
6. API & 데이터베이스 레퍼런스
7. API Reference (외부/내부)
8. Changelog

Chapter 1

CSO 정산 포털

Next.js TypeScript Supabase Tailwind CSS Netlify

CSO 정산 포털

`v0.16.0`

CSO(위탁영업) 업체가 제약사 정산 수수료를 조회하는 B2B 포털

아키텍처 · API 레퍼런스 · 운영 가이드 · 이관 매뉴얼

▶ 목차

프로젝트 소개

제약사가 CSO(위탁영업) 업체에 지급하는 정산 수수료를 관리하는 포털입니다.

왜 만들었는가:

- 제약사 정산 데이터(SIT솔루션 Compare&Chart 다운로드 엑셀)를 수작업으로 각 CSO 업체에 개별 전달하는 비효율을 없애기 위해
- CSO 업체가 직접 로그인해서 자사 정산 데이터만 조회·다운로드할 수 있도록
- 업체명 ↔ 사업자번호 매칭을 체계적으로 관리하여 정산 누락을 방지하기 위해

핵심 메커니즘:

개념	설명
사업자등록번호 식별	B2B이므로 이메일이 아닌 사업자번호가 사용자를 구분하는 핵심 키
CSO 매칭	엑셀 속 텍스트 업체명 ↔ 회원 사업자번호를 연결하는 매핑 테이블
한글 컬럼명	제약업계 엑셀 양식 그대로 반영 (47개 컬럼)

사용자 역할:

역할	할 수 있는 일
관리자	SIT솔루션 Compare&Chart 엑셀 업로드, 회원 승인/관리, CSO 매칭 설정, 메일 발송
일반 회원	자사 사업자번호에 해당하는 정산 데이터 조회/다운로드

기술 스택

기술	역할
Next.js	App Router 기반 풀스택 프레임워크 (v14)
TypeScript	타입 안전성 (strict 모드)
Supabase	PostgreSQL + REST API (인증은 자체 JWT)
Tailwind CSS	유틸리티 기반 스타일링
Netlify	자동 배포 (GitHub → main push)

주요 라이브러리: Radix UI (컴포넌트), exceljs/xlsx (엑셀 파싱), jose (JWT), bcryptjs (해싱), resend (이메일), react-dropzone (업로드)

테스트: Vitest (유닛/통합/컴포넌트), Playwright (E2E), Testing Library (React 컴포넌트)

시작하기

로컬 환경에서 프로젝트를 실행하는 방법입니다. 상세한 가이드는 ONBOARDING을 참고하세요.

사전 요구사항

- Node.js 18+
- npm
- Git
- Supabase 프로젝트 접근 권한 (기존 팀원에게 대시보드 초대 필요)

설치

1. **저장소 클론**

```
git clone https://github.com/<org>/csoweb.git
cd csoweb
```

2. **의존성 설치**

```
npm install
```

3. **환경변수 설정**

```
cp .env.local.example .env.local
```

`.env.local`을 열어 Supabase URL, 키, JWT 시크릿 등 8개 변수를 채웁니다. 각 변수의 설명과 획득 경로는 ONBOARDING을 참고하세요.

4. **개발 서버 실행**

```
npm run dev
```

`http://localhost:3000`에서 로그인 화면이 나타나면 성공입니다.

5. **테스트 계정 설정**

회원가입 후 Supabase 대시보드에서 `is_approved = true`로 변경하거나, 기존 테스트 계정을 사용합니다. 첫 로그인 시 비밀번호 변경이 강제됩니다.

사용법

관리자 워크플로우

```
SIT솔루션 Compare&Chart 엑셀 업로드 → 미리보기 확인 → DB 저장 확정
  → CSO 매칭 관리 (업체명 ↔ 사업자번호)
  → 회원 승인/관리
  → 정산 안내 메일 발송 (메일머지)
```

일반 회원 워크플로우

```
로그인 → 정산 데이터 조회 (월별 필터)
  → 엑셀 다운로드
  → 월별 합계 확인
```

자세한 API 엔드포인트는 API-DATABASE를, 코드 구조는 ARCHITECTURE를 참고하세요.

테스트

```
npm test              # 전체 유닛+통합+컴포넌트 (171개 케이스)
npm run test:watch    # 워치 모드
npm run test:unit     # 유닛만 (lib/, application/)
```

```
npm run test:integration  # API 통합만 (app/api/)
npm run test:component    # 컴포넌트만 (.test.tsx)
npm run test:coverage     # 커버리지 리포트 (v8)
npm run test:ui           # Vitest UI (브라우저)
npm run test:e2e          # Playwright E2E (dev server 자동 실행)
npm run test:e2e:headed   # Playwright headed 모드
```

카테고리	파일 수	케이스 수	대상
Unit	3	45	`lib/auth`, `lib/utils`, `lib/excel`
Application Use Case	9	51	`application/user/*`, `application/settlement/*`, `application/cso-matching/*`, `application/auth/*`
API 통합	13	63	`api/auth/*`, `api/users/*`, `api/settlements/*`, `api/columns`
컴포넌트	2	12	`AuthContext`, `Header`
합계	**27**	**171**	

로드맵

- [x] 회원가입 / 로그인 / 비밀번호 관리
- [x] 관리자 회원 승인/거부 흐름
- [x] SIT솔루션 Compare&Chart 엑셀 업로드 (미리보기 + 확정)
- [x] CSO 매칭 (업체명 ↔ 사업자번호)
- [x] 정산 데이터 조회 / 엑셀 다운로드
- [x] 월별 합계 및 통계
- [x] 메일머지 (정산 안내 일괄 발송)
- [x] 컬럼 표시 설정 (가시성, 순서)
- [x] CSO 매칭 무결성 검사
- [x] 회원가입 사업자번호 국세청 실시간 인증 (계속사업자만 가입 허용)
- [x] 이메일 듀얼 프로바이더 (Resend + SMTP 하이웍스)
- [x] 메일머지 SSE 실시간 진행률 + 수신자 수 표시
- [x] 대시보드 시스템 정보 강화 (SMTP/NTS 상태, 버전 동기화)
- [x] 정산서 Notice 편집을 마스터조회 페이지로 이동 (사용 맥락에서 인라인 편집)
- [ ] CSO 필터링
- [x] DDD 레이어 전환 완료 (`lib/db.ts` 호환 레이어 제거)
- [x] 테스트 인프라 구축 (Vitest + Playwright, 108개 케이스)
- [ ] SIT솔루션 API 연동 (엑셀 수동 업로드 → API 자동 통신 전환)
- [ ] 다국어 지원

기여 방법

프로젝트에 기여하는 방법입니다.

1. 저장소 Fork
2. Feature 브랜치 생성 (`git checkout -b feature/amazing-feature`)
3. 변경사항 커밋 (`git commit -m 'feat: 기능 설명'`)
4. 브랜치에 Push (`git push origin feature/amazing-feature`)
5. Pull Request 생성

코드 작성 시 DDD 레이어 구조를 따라주세요. 자세한 내용은 ARCHITECTURE를 참고하세요.

라이선스

비공개 프로젝트입니다. 무단 배포를 금지합니다.

연락처

영업관리팀 권대환 — qwer@ukp.co.kr

참고 자료

- Next.js Documentation
- Supabase Documentation
- Tailwind CSS Documentation
- Radix UI
- Resend Email API
- Best-README-Template

Chapter 2

이관 매뉴얼

프로젝트 담당자 변경 또는 인프라를 새 환경으로 이관할 때 사용하는 절차서입니다.

> **관련 문서**
>
> - 로컬 환경 세팅 → ONBOARDING.md
> - 일상 운영/트러블슈팅 → OPERATIONS.md
> - API/DB 스키마 → API-DATABASE.md

목차

1. 사전 준비
2. GitHub 이관
3. Supabase 이관
4. Netlify 이관
5. 외부 서비스 설정
6. 이관 후 검증 체크리스트
7. 환경변수 전체 레퍼런스
8. 코드 레벨 변경사항
9. 다음(Daum) 주소 API
10. 초기화 스크립트
11. 커스텀 도메인 / DNS

Part 1: 사전 준비

필요 계정 및 권한

서비스	필요 권한	비고
GitHub	Repository Admin (transfer 시) 또는 Fork 권한	조직 저장소면 Owner 권한 필요
Supabase	Organization Owner (transfer 시) 또는 새 프로젝트 생성 권한	Free Tier 사용 중
Netlify	Team Owner (transfer 시) 또는 새 사이트 생성 권한	
Resend	계정 + 도메인 인증	이메일 발송용

서비스	필요 권한	비고
data.go.kr	계정 + API 활용신청 승인	국세청 NTS API, HIRA API
Vercel (선택)	프로젝트 접근 권한	개발자 테스트 배포용 병행 사용 중

현재 환경 정보 수집 체크리스트

이관 전 현재 환경에서 다음 정보를 수집합니다:

- [ ] Supabase 프로젝트 URL (`https://xxxxx.supabase.co`)
- [ ] Supabase API 키 3종 (URL, anon key, service role key)
- [ ] Netlify 사이트 URL 및 사이트 이름
- [ ] Netlify 환경변수 10개 값 (Part 7 참조)
- [ ] GitHub Secrets 2개 (`SUPABASE_URL`, `SUPABASE_ANON_KEY`)
- [ ] Resend API 키 및 인증 도메인
- [ ] NTS API 키 (data.go.kr)
- [ ] HIRA API 키 (data.go.kr, 선택)
- [ ] 커스텀 도메인 DNS 레코드 (사용 시)
- [ ] 현재 JWT_SECRET 값
- [ ] SMTP 설정 (DB `company_settings` 테이블에 저장됨 — 관리자 대시보드에서 확인)
- [ ] Vercel 프로젝트 URL (사용 시)

Part 2: GitHub 이관

방법 A: Repository Transfer (권장)

GitHub Settings → Danger Zone → Transfer repository로 새 소유자/조직에 이전합니다.

- 이슈, PR, 위키, Stars, 설정이 모두 이전됨
- Collaborators는 수동으로 재초대 필요

방법 B: Fork

원본 저장소를 Fork하여 독립 저장소로 운영합니다.

GitHub Secrets 설정

Repository → Settings → Secrets and variables → Actions에서 2개 설정:

Secret 이름	값	용도
`SUPABASE_URL`	`https://xxxxx.supabase.co`	keep-alive workflow
`SUPABASE_ANON_KEY`	Supabase anon key	keep-alive workflow

GitHub Actions 확인

`.github/workflows/keep-supabase-alive.yml` 이 정상 동작하는지 확인합니다:

- **스케줄**: 2시간마다 (`cron: '0 0 */2 * *'`)
- **동작**: Supabase REST API에 `users?select=id&limit=1` 요청
- **목적**: Free Tier Supabase의 7일 미활동 자동 휴면 방지

Actions 탭에서 수동 실행(workflow_dispatch)으로 테스트:

```
GitHub → Actions → "Keep Supabase Alive" → Run workflow
```

Branch Protection Rules

현재 별도의 브랜치 보호 규칙이 설정되어 있지 않습니다. 필요에 따라 `main` 브랜치에 보호 규칙을 설정하세요.

Part 3: Supabase 이관

방법 A: 소유권 이전 (Organization Transfer)

같은 Supabase 프로젝트를 유지하면서 조직/소유자만 변경하는 방법입니다.

1. Supabase 대시보드 → Organization Settings → General
2. Transfer project to another organization
3. 새 소유자가 Organization에 초대되어 있어야 함

이 경우 URL, API 키가 변경되지 않으므로 환경변수 수정이 불필요합니다.

방법 B: 새 프로젝트 생성 후 데이터 마이그레이션

1. 새 프로젝트 생성

Supabase 대시보드에서 새 프로젝트를 생성합니다.

- Region: `Northeast Asia (Tokyo)` 권장 (Netlify Functions 리전과 동일)
- Database password를 안전하게 보관

2. 스키마 생성 순서

테이블 간 의존성이 있으므로 아래 순서로 생성합니다:

```
1. users                    ← 기본 사용자 테이블
2. cso_matching             ← users.business_number 참조
3. settlements              ← users.business_number 참조
4. column_settings          ← 독립 (전역 설정)
5. company_settings         ← 독립 (전역 설정, DB상 company_info)
6. email_logs               ← 독립 (감사 로그)
7. password_reset_tokens    ← users.email 참조
8. hospital_master          ← 독립 (선택, Edge Function용)
```

> DB 스키마 상세는 API-DATABASE.md를 참조하세요.

3. SQL 마이그레이션 파일 적용 순서

`supabase/migrations/` 디렉토리의 파일을 타임스탬프 순서대로 적용합니다:

```
1. 20240209_setup_hospital_sync_cron.sql    ← pg_cron/pg_net 확장, 병원 동기화 크론
2. 20260218_fix_cso_matching_rls.sql        ← cso_matching RLS 정책 추가
3. 20260218_fix_rls_auth_initialization.sql ← RLS 성능 최적화 (4개 테이블)
4. 20260225_add_account_lockout.sql         ← users 테이블 잠금 컬럼 추가
```

Supabase SQL Editor에서 순서대로 실행하거나, Supabase CLI를 사용합니다:

```
supabase db push
```

4. RLS 정책 설정

모든 테이블에 service_role 전체 접근 정책이 필요합니다. 앱은 서버 사이드에서 `SUPABASE_SERVICE_ROLE_KEY`로 접근하므로 RLS를 우회합니다.

마이그레이션 파일 2, 3번에서 아래 테이블의 정책이 설정됩니다:

- `cso_matching`
- `company_settings`
- `hospital_master`
- `hospital_sync_logs`
- `password_reset_tokens`

나머지 테이블(`users`, `settlements`, `column_settings`, `email_logs`)도 RLS가 활성화되어 있다면 동일한 정책을 추가합니다:

```
CREATE POLICY "Service role full access" ON public.<table_name>
  USING ((select auth.role()) = 'service_role')
  WITH CHECK ((select auth.role()) = 'service_role');
```

5. 데이터 Export/Import

pg_dump/pg_restore 방식:

```
# 기존 DB에서 export
pg_dump -h <old-host> -U postgres -d postgres \
  --data-only --no-owner --no-privileges \
  -t users -t cso_matching -t settlements \
  -t column_settings -t company_info -t email_logs \
  -t password_reset_tokens \
  > data_dump.sql

# 새 DB에 import
psql -h <new-host> -U postgres -d postgres < data_dump.sql
```

Supabase CLI 방식:

```
# 기존 프로젝트에서
supabase db dump --data-only -f data_dump.sql

# 새 프로젝트에
psql "postgresql://postgres:<password>@<new-host>:5432/postgres" < data_dump.sql
```

> Supabase 연결 정보는 대시보드 → Settings → Database에서 확인합니다.

6. Edge Function 배포 (sync-hospitals)

```
# Supabase CLI 로그인
supabase login

# 프로젝트 연결
supabase link --project-ref <new-project-ref>

# Edge Function 배포
supabase functions deploy sync-hospitals
```

Edge Function은 `supabase/functions/sync-hospitals/` 디렉토리에 있으며, 건강보험심사평가원 API에서 병원 마스터 데이터를 동기화합니다.

> 참고: Supabase Storage는 미사용 (엑셀 파일은 메모리 파싱 후 DB 저장). Storage 이관은 불필요합니다.

7. API 키 확보

새 프로젝트 대시보드 → Settings → API에서 3개 키를 확보합니다:

키	환경변수	용도
Project URL	`NEXT_PUBLIC_SUPABASE_URL`	Supabase 접속 URL

키	환경변수	용도
anon (public) key	`NEXT_PUBLIC_SUPABASE_ANON_KEY`	클라이언트용 공개 키
service_role key	`SUPABASE_SERVICE_ROLE_KEY`	서버용 비밀 키 (RLS 우회)

Part 4: Netlify 이관

방법 A: 소유권 이전

1. Netlify Team Settings → Members에서 새 담당자를 Owner로 초대
2. 기존 Owner 권한을 조정하거나 제거

이 경우 사이트 URL, 환경변수, 배포 설정이 모두 유지됩니다.

방법 B: 새 사이트 생성

1. GitHub 연동

Netlify → Add new site → Import an existing project → GitHub

- Repository: `csoweb`
- Branch to deploy: `main`
- Build command: `npm run build`
- Publish directory: `.next`

2. 환경변수 설정 (10개)

Site settings → Environment variables에서 설정:

변수	값 예시	필수
`NEXT_PUBLIC_SUPABASE_URL`	`https://xxxxx.supabase.co`	O
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	`eyJhbGciOiJ...`	O
`SUPABASE_SERVICE_ROLE_KEY`	`eyJhbGciOiJ...`	O
`JWT_SECRET`	(32자 이상 랜덤 문자열)	O
`RESEND_API_KEY`	`re_xxxxxxxx`	O
`EMAIL_FROM`	`CSO Portal <noreply@your-domain.com>`	O
`ADMIN_EMAIL`	`admin@your-domain.com`	O
`NEXT_PUBLIC_BASE_URL`	`https://your-production-url.com`	O

변수	값 예시	필수
`NTS_API_KEY`	data.go.kr 발급 키	O
`HIRA_API_KEY`	data.go.kr 발급 키	선택

> `JWT_SECRET`은 기존 환경과 동일한 값을 사용해야 기존 사용자 세션이 유효합니다. 새 프로젝트라면 새로 생성해도 됩니다.

3. `netlify.toml` 확인

```
[build]
  command = "npm run build"
  publish = ".next"

[build.environment]
  NODE_VERSION = "20"

[functions]
  region = "ap-northeast-1"    # 도쿄 (Supabase 리전과 가까운 곳)

[[plugins]]
  package = "@netlify/plugin-nextjs"
```

파일이 저장소에 포함되어 있으므로 별도 수정은 불필요합니다.

4. 커스텀 도메인 설정

커스텀 도메인을 사용하는 경우:

1. Netlify → Domain settings → Add custom domain
2. DNS 제공업체에서 CNAME 또는 A 레코드 설정
3. Netlify가 자동으로 SSL 인증서(Let's Encrypt) 발급

> 자세한 내용은 Part 11: 커스텀 도메인 / DNS를 참조하세요.

5. 배포 트리거 확인

- GitHub `main` 브랜치에 push하면 Netlify가 자동 빌드/배포
- Netlify 대시보드 → Deploys → Trigger deploy → Deploy site로 수동 배포 가능

Part 5: 외부 서비스 설정

Resend (이메일 API)

1. resend.com 회원가입
2. Settings → API Keys → Create API Key

3. Domains → Add Domain → DNS 레코드 추가 (SPF, DKIM)
4. 도메인 인증 완료 후 실제 발신 가능

발급받은 API 키를 `RESEND_API_KEY` 환경변수에 설정합니다.

> 도메인 인증 전에는 `onboarding@resend.dev`에서만 발신 가능 (테스트용).

국세청 NTS API

사업자등록번호 실시간 인증에 사용합니다 (`/api/verify-biz`).

1. data.go.kr 회원가입
2. 국세청 사업자등록정보 진위확인 및 상태조회 서비스 활용신청
3. 승인 후 마이페이지에서 API 키 확인
4. `NTS_API_KEY` 환경변수에 설정

> API 엔드포인트: `https://api.odcloud.kr/api/nts-businessman/v1/status`

건강보험심사평가원 HIRA API (선택)

병원 마스터 데이터 동기화에 사용합니다 (Edge Function `sync-hospitals`).

1. data.go.kr 에서 활용신청
   - 병원정보서비스
   - 약국정보서비스
2. 승인 후 API 키 확인
3. `HIRA_API_KEY` 환경변수에 설정

> HIRA API를 사용하지 않으면 병원 자동완성 기능이 비활성화됩니다. 나머지 기능에는 영향 없습니다.

SMTP (선택)

Resend 외에 자체 SMTP 서버로 이메일을 발송할 수 있습니다.

- 환경변수가 아닌 **DB(`company_settings` 테이블)에 저장**
- 관리자 대시보드 → 사이트 설정 → 이메일 발송 설정에서 구성
- 프로바이더를 `smtp`로 변경하면 SMTP 발송으로 전환
- 하이웍스 예시: `smtps.hiworks.com:465` (SSL)

Part 6: 이관 후 검증 체크리스트

빌드 & 테스트

- [ ] `npm run build` — 빌드 성공 확인

- ☐ `npm test` — 27개 파일, 171개 테스트 케이스 통과 확인

기능 검증

- ☐ **로그인** — 사업자등록번호 + 비밀번호로 로그인
- ☐ **회원가입** — 사업자번호 국세청 인증 → 가입 신청 → 관리자 승인 → 로그인
- ☐ **정산 데이터 조회** — 월별 필터, 페이징, 검색 정상 동작
- ☐ **정산 엑셀 다운로드** — 다운로드 파일 열기 확인
- ☐ **이메일 발송** — 관리자 → 사이트 설정 → 이메일 발송 설정 → 테스트 발송
- ☐ **CSO 매칭** — 관리자 → CSO 매칭 관리에서 매핑 확인

인프라 검증

- ☐ **Netlify 배포** — `main` push 후 자동 배포 트리거 확인
- ☐ **GitHub Actions** — keep-alive workflow 수동 실행 후 성공 확인
- ☐ **시스템 상태** — `/api/system/status` 엔드포인트로 전체 상태 확인
  - Supabase 연결 상태
  - NTS API 키 설정 여부
  - HIRA API 키 설정 여부
  - 이메일 프로바이더 상태 (Resend/SMTP)

보안 검증

- ☐ 환경변수에 실제 키 값이 설정되었는지 확인 (placeholder 아닌 실제 값)
- ☐ `SUPABASE_SERVICE_ROLE_KEY` 가 클라이언트에 노출되지 않는지 확인 (`NEXT_PUBLIC_` 접두사 없음)
- ☐ HTTPS 적용 여부 확인 (커스텀 도메인 사용 시)

Part 7: 환경변수 전체 레퍼런스

필수 환경변수 (10개)

#	변수	용도	획득 경로	보안등급	설정 위치
1	`NEXT_PUBLIC_SUPABASE_URL`	Supabase 프로젝트 URL	Supabase 대시보드 → Settings → API	공개	Netlify + `.env.local` + GitHub Secrets(`SUPABASE_URL`)

#	변수	용도	획득 경로	보안등급	설정 위치
2	`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase 공개 키 (클라이언트용)	같은 위치	공개	Netlify + `.env.local` + GitHub Secrets(`SUPABASE_ANON_K`
3	`SUPABASE_SERVICE_ROLE_KEY`	Supabase 서비스 키 (RLS 우회)	같은 위치	**비밀**	Netlify + `.env.local`
4	`JWT_SECRET`	JWT 서명 키 (HS256, 32자 이상)	자체 생성 (`openssl rand -base64 48`)	**비밀**	Netlify + `.env.local`
5	`RESEND_API_KEY`	Resend 이메일 발송 API 키	resend.com → API Keys	**비밀**	Netlify + `.env.local`
6	`EMAIL_FROM`	발신자 이메일 주소	형식: `이름 <email@domain>`	일반	Netlify + `.env.local`
7	`ADMIN_EMAIL`	관리자 알림 수신 이메일	관리자 이메일 주소	일반	Netlify + `.env.local`
8	`NEXT_PUBLIC_BASE_URL`	사이트 기본 URL (이메일 링크에 사용)	프로덕션 도메인	공개	Netlify + `.env.local`
9	`NTS_API_KEY`	국세청 사업자등록 정보 API 키	data.go.kr	**비밀**	Netlify + `.env.local`
10	`HIRA_API_KEY`	건강보험심사평가원 병원/약국 API 키	data.go.kr	**비밀**	Netlify + `.env.local`

선택 환경변수

변수	용도	비고
`HIRA_API_KEY`	병원 마스터 동기화	미설정 시 병원 자동완성 비활성화, 나머지 기능 정상

> `NEXT_PUBLIC_` 접두사가 붙은 변수만 브라우저에 노출됩니다. 나머지는 서버(Netlify Functions)에서만 접근 가능합니다.

GitHub Secrets (Netlify 환경변수와 별도)

Secret 이름	대응하는 환경변수	용도
`SUPABASE_URL`	`NEXT_PUBLIC_SUPABASE_URL`과 동일한 값	keep-alive workflow
`SUPABASE_ANON_KEY`	`NEXT_PUBLIC_SUPABASE_ANON_KEY`와 동일한 값	keep-alive workflow

Part 8: 코드 레벨 변경사항

Supabase 프로젝트를 새로 생성한 경우(방법 B), 아래 코드 레벨 확인이 필요합니다.

CSP 도메인 확인

`src/middleware.ts`의 CSP 설정에서 `connect-src` 지시문을 확인합니다:

```
// src/middleware.ts:75
"connect-src 'self' https://*.supabase.co",
```

현재 `https://*.supabase.co` 와일드카드를 사용하므로, Supabase 프로젝트 URL이 `*.supabase.co` 형태면 변경 불필요합니다. 커스텀 도메인을 사용하는 경우 해당 도메인을 추가하세요.

`NEXT_PUBLIC_BASE_URL` 변경

이메일 본문의 링크(로그인, 비밀번호 재설정 등)에 사용됩니다.

```
// src/lib/email.ts:17
const BASE_URL = process.env.NEXT_PUBLIC_BASE_URL || 'http://localhost:3000';
```

프로덕션 도메인이 변경되면 Netlify 환경변수의 `NEXT_PUBLIC_BASE_URL`을 신규 도메인으로 반드시 업데이트하세요.

Vercel (테스트 배포)

빌드 시간 여유로 개발자 테스트용으로 Vercel을 병행 사용 중입니다.

- `@vercel/speed-insights` 패키지가 의도적으로 유지되어 있음 (`src/app/layout.tsx`)

- 이관 시 Vercel 프로젝트 접근 권한도 함께 이전 필요
- Vercel을 사용하지 않을 경우:
  i. `npm uninstall @vercel/speed-insights`
  ii. `src/app/layout.tsx`에서 `<SpeedInsights />` 컴포넌트 제거

Part 9: 다음(Daum) 주소 API

회원가입과 프로필 완성 페이지에서 주소 검색에 사용합니다.

- 사용 위치:
  - `src/app/(auth)/register/page.tsx` — 회원가입
  - `src/app/(auth)/complete-profile/page.tsx` — 프로필 완성
- CSP에 `https://t1.daumcdn.net`이 `script-src`에 허용되어 있음
- **API 키 없이 동작** (무료 공개 API)
- 이관 시 특별한 작업 없음 — 동작 확인만 하면 됩니다

Part 10: 초기화 스크립트

새 DB를 초기화하거나 데이터를 세팅할 때 사용하는 스크립트입니다.

스크립트 목록

스크립트	용도	실행 조건
`scripts/hospital_master_ddl.sql`	병원 마스터 테이블 DDL	새 DB에서 hospital_master 테이블 필요 시
`scripts/upload_hospital_data.ts`	병원 데이터 벌크 업로드	hospital_master 초기 데이터 적재 시
`scripts/seed-users.ts`	CSO 매칭 기반 사용자 일괄 생성	테스트 계정 또는 초기 사용자 대량 생성 시
`scripts/rehash-passwords.ts`	비밀번호 정책 변경 시 재해싱	bcrypt cost 변경 등 보안 정책 업데이트 시

신규 DB 초기화 실행 순서

1. 스키마 생성 (Part 3의 테이블 생성 순서 참조)
2. SQL 마이그레이션 4개 적용
3. `scripts/hospital_master_ddl.sql` 실행 (병원 마스터 필요 시)
4. `scripts/upload_hospital_data.ts` 실행 (병원 데이터 적재)
5. `scripts/seed-users.ts` 실행 (초기 사용자 생성, 필요 시)

6. 기존 데이터가 있다면 pg_dump/pg_restore로 이관 (Part 3 참조)

> 스크립트 실행 시 `.env.local`에 Supabase 환경변수가 설정되어 있어야 합니다.

Part 11: 커스텀 도메인 / DNS

현재 상태

- `netlify.toml`에는 도메인 설정이 없음 (Netlify 대시보드에서 관리)
- 커스텀 도메인을 사용하는 경우 Netlify 대시보드에서 설정됨

이관 절차

1. **기존 DNS 레코드 캡처**: 현재 도메인의 DNS 설정을 기록
   - CNAME 레코드 (예: `www` → `xxx.netlify.app`)
   - A 레코드 (필요 시)
   - 기타 TXT 레코드 (이메일 인증용 SPF, DKIM 등)
2. **새 Netlify 사이트에 도메인 추가**:
   - Site settings → Domain management → Add custom domain
   - Netlify가 자동으로 Let's Encrypt SSL 인증서 발급
3. **DNS 레코드 변경**:
   - DNS 제공업체에서 CNAME/A 레코드를 새 Netlify 사이트로 변경
   - TTL이 낮으면(300초) 빠르게 전파됨
4. **`NEXT_PUBLIC_BASE_URL` 업데이트**:
   - 도메인이 변경되면 Netlify 환경변수의 `NEXT_PUBLIC_BASE_URL`도 업데이트
   - 이메일 본문 링크가 올바른 도메인을 가리키는지 확인
5. **Resend 도메인 인증** (해당 시):
   - 도메인이 변경되면 Resend에서 새 도메인 인증 필요
   - DNS에 SPF, DKIM 레코드 추가

참고사항

Supabase Storage

현재 미사용입니다. 엑셀 파일은 업로드 시 메모리에서 파싱한 후 DB에 저장하므로 Storage 이관은 불필요합니다.

기존 문서 참조

작업	참조 문서
로컬 개발 환경 세팅	ONBOARDING.md
이관 후 일상 운영/트러블슈팅	OPERATIONS.md
API 엔드포인트/DB 스키마 상세	API-DATABASE.md
코드 아키텍처	ARCHITECTURE.md

Chapter 3

온보딩 가이드

로컬 환경에서 프로젝트를 실행하기까지의 과정입니다.

사전 요구사항

- **Node.js 18+** (`node -v`로 확인)
- **npm** (Node.js 설치 시 포함)
- **Git**
- **Supabase 프로젝트 접근 권한** — 기존 팀원에게 대시보드 초대를 받아야 합니다

1단계: 클론 + 의존성 설치

```
git clone https://github.com/<org>/csoweb.git
cd csoweb
npm install
```

2단계: 환경변수 설정

`.env.local.example`을 복사하여 `.env.local`을 만듭니다.

```
cp .env.local.example .env.local
```

각 변수를 채워 넣습니다:

변수	설명	획득 경로
`NEXT_PUBLIC_SUPABASE_URL`	Supabase 프로젝트 URL	Supabase 대시보드 → Settings → API
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase 공개 키 (클라이언트용)	같은 위치
`SUPABASE_SERVICE_ROLE_KEY`	Supabase 서비스 키 (서버용, RLS 우회)	같은 위치 — **절대 클라이언트에 노출 금지**
`JWT_SECRET`	JWT 서명 키 (32자 이상)	기존 팀원에게 전달받거나, 로컬 전용으로 새로 생성
`RESEND_API_KEY`	Resend 이메일 발송 API 키	resend.com 대시보드
`EMAIL_FROM`	발신자 이메일 주소	예: `CSO Portal <noreply@example.com>`

변수	설명	획득 경로
`ADMIN_EMAIL`	관리자 알림 수신 이메일	관리자 이메일 주소
`NEXT_PUBLIC_BASE_URL`	사이트 기본 URL	로컬: `http://localhost:3000`

> **참조**: `.env.local.example` 파일에 전체 변수 목록이 있습니다.

3단계: 개발 서버 실행

```
npm run dev
```

`http://localhost:3000` 에서 로그인 화면이 나타나면 성공입니다.

4단계: 테스트 로그인

로컬 DB에 테스트 계정이 있다면 그대로 사용합니다. 없다면:

1. `/register` 에서 회원가입 (사업자등록번호 10자리 필요)
2. Supabase 대시보드 → `users` 테이블에서 해당 계정의 `is_approved` 를 `true` 로 변경
3. 관리자 계정이 필요하면 `is_admin` 도 `true` 로 변경

첫 로그인 시 비밀번호 강제 변경 화면이 나타납니다. `must_change_password` 가 `true` 인 계정은 비밀번호를 변경해야 메인 화면에 진입할 수 있습니다.

자주 겪는 문제

증상	원인	해결
`fetch failed` / Supabase 연결 실패	`.env.local` 의 Supabase URL 또는 키가 잘못됨	대시보드에서 값 재확인, `NEXT_PUBLIC_` 접두사 확인
빌드 에러 (`npm run build` 실패)	TypeScript 타입 에러 또는 의존성 누락	`npm install` 재실행, 에러 메시지의 파일 확인
로그인 후 빈 화면	JWT_SECRET이 서버와 불일치	`.env.local` 의 JWT_SECRET 확인
회원가입 후 로그인 불가	관리자 승인 대기 상태	DB에서 `is_approved = true` 설정
`Module not found` 에러	`@/` 경로 별칭 인식 실패	`tsconfig.json` 의 paths 설정 확인, IDE 재시작

Chapter 4

배포 & 운영

배포 파이프라인, 인프라 구성, 문제 해결 방법을 다룹니다.

배포 파이프라인

GitHub `main` 브랜치에 push하면 Netlify가 자동으로 빌드·배포합니다.

```
git push origin main  →  Netlify 감지  →  npm run build  →  배포 완료
```

빌드 설정은 `netlify.toml`에 정의되어 있습니다:

- 빌드 명령: `npm run build`
- Node 버전: 18
- Functions 리전: `ap-northeast-1` (도쿄)
- 플러그인: `@netlify/plugin-nextjs`

> 참조: `netlify.toml`

배포 전 체크리스트

1. `npm run build` — 로컬에서 빌드 성공 확인
2. `npm run lint` — lint 에러 없는지 확인
3. Netlify 대시보드에서 환경변수가 최신인지 확인 (특히 새 변수 추가 시)

환경변수 관리

변수	용도	설정 위치
`NEXT_PUBLIC_SUPABASE_URL`	Supabase 프로젝트 URL	Netlify + `.env.local`
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase 공개 키	Netlify + `.env.local`
`SUPABASE_SERVICE_ROLE_KEY`	Supabase 서비스 키 (RLS 우회)	Netlify + `.env.local`
`JWT_SECRET`	JWT 서명 키	Netlify + `.env.local`
`RESEND_API_KEY`	Resend 이메일 API 키	Netlify + `.env.local`
`EMAIL_FROM`	발신자 이메일	Netlify + `.env.local`

변수	용도	설정 위치
`ADMIN_EMAIL`	관리자 알림 수신 이메일	Netlify + `.env.local`
`NEXT_PUBLIC_BASE_URL`	사이트 기본 URL	Netlify(프로덕션 URL) + `.env.local`(localhost)

`NEXT_PUBLIC_` 접두사가 붙은 변수만 클라이언트(브라우저)에 노출됩니다. 나머지는 서버에서만 접근 가능합니다.

인프라 구성 요소

Supabase (데이터베이스)

- **역할**: PostgreSQL 데이터베이스 + REST API
- **인증**: Supabase Auth는 사용하지 않음 (자체 JWT)
- **RLS**: 일부 테이블에 적용됨. 서비스 키 사용 시 RLS 우회
- **플랜**: Free Tier (7일 미활동 시 자동 휴면 — 아래 워크플로우로 방지)

GitHub Actions: Supabase 휴면 방지

Free Tier Supabase는 7일간 활동이 없으면 자동 일시정지됩니다. 이를 방지하기 위해 2시간마다 간단한 쿼리를 보내는 워크플로우가 동작합니다.

- **스케줄**: 2시간마다 (`cron: '0 0 */2 * *'`)
- **동작**: Supabase REST API에 `users?select=id&limit=1` 요청
- **Secrets**: `SUPABASE_URL`, `SUPABASE_ANON_KEY` (GitHub repository secrets)

> 참조: `.github/workflows/keep-supabase-alive.yml`

Supabase Edge Function: sync-hospitals

병원 마스터 데이터를 건강보험심사평가원 API에서 동기화하는 Edge Function입니다.

- **호출**: POST 요청 (인증 헤더 또는 크론 헤더 필요)
- **동작**: 공공 API에서 병원 정보를 페이지 단위로 가져와 `hospital_master` 테이블에 upsert
- **설정**: 1000건/페이지, 500건/배치, 3회 재시도

> 참조: `supabase/functions/sync-hospitals/`

트러블슈팅

증상별 대응표

증상	원인	확인 방법	해결
로그인 불가 — "승인 대기"	관리자 미승인	DB `users` 테이블에서 `is_approved` 확인	관리자가 승인 처리
로그인 불가 — 비밀번호 오류	비밀번호 불일치 또는 초기화됨	관리자가 비밀번호 초기화 여부 확인	비밀번호 재설정 또는 관리자 초기화
정산 데이터 0건	CSO 매칭 누락	관리자 → CSO 매칭 관리에서 무결성 검사	해당 업체의 매칭 추가
업로드 실패	SIT솔루션 Compare&Chart 엑셀 형식 불일치 (컬럼명, 시트 구조)	에러 메시지에서 실패 행/컬럼 확인	SIT솔루션에서 엑셀 재다운로드 후 업로드
빌드 실패 (Netlify)	TypeScript 에러 또는 의존성 문제	Netlify 대시보드 → Deploy → 빌드 로그	로컬 `npm run build`로 재현 후 수정
이메일 미발송	Resend API 키 만료 또는 일일 한도 초과	`/api/email/logs`에서 상태 확인	Resend 대시보드에서 키 재발급
Supabase 503 에러	Free Tier 자동 휴면	Supabase 대시보드에서 프로젝트 상태 확인	대시보드에서 수동 복원 (몇 분 소요)
정산 금액 불일치	업로드 시 숫자 파싱 오류	원본 엑셀과 DB 데이터 대조	해당 월 삭제 후 재업로드

로그 확인 경로

로그 종류	확인 경로
빌드 로그	Netlify 대시보드 → Deploys → 해당 배포 클릭
런타임 로그 (API 에러 등)	Netlify 대시보드 → Logs → Functions
DB 쿼리 로그	Supabase 대시보드 → Logs → Postgres
이메일 발송 이력	앱 내 `/api/email/logs` 또는 Resend 대시보드
시스템 상태	`/api/system/status` API 호출
Edge Function 로그	Supabase 대시보드 → Edge Functions → sync-hospitals → Logs

Chapter 5

아키텍처

이 문서는 csoweb의 비즈니스 도메인과 코드 구조를 설명합니다. "왜 이렇게 만들었는가"에 초점을 맞춥니다.

비즈니스 도메인

전체 흐름

```
관리자가 SIT솔루션 Compare&Chart 엑셀 업로드
  → 파싱 후 DB 저장
  → CSO 매칭으로 업체별 데이터 분류
  → 일반 회원은 자사 정산만 조회/다운로드
```

핵심 개념 3가지

1. 사업자등록번호 = 사용자 식별자

B2B 서비스이므로 개인 이메일이 아닌 **법인 사업자등록번호**가 사용자를 구분하는 핵심 키입니다. 하나의 사업자번호에 하나의 계정이 매핑되며, 정산 데이터 조회 범위도 이 번호로 결정됩니다.

2. CSO 매칭 — 이 시스템의 가장 독자적인 메커니즘

제약사 정산 엑셀에는 CSO 업체의 **텍스트 이름**만 들어있고, 사업자번호는 없습니다. 따라서 "엑셀 속 업체명 ↔ 회원의 사업자번호"를 연결하는 매핑 테이블이 필요합니다. 이것이 CSO 매칭입니다.

```
엑셀: "한국CSO"  →  cso_matching 테이블  →  사업자번호 "1234567890"  →  회원 조회
```

매칭이 없으면 해당 정산 데이터는 어떤 회원에게도 보이지 않습니다. 관리자가 매칭을 관리하며, 무결성 검사(integrity check) 기능으로 누락된 매칭을 찾을 수 있습니다.

> 참조: `src/domain/cso-matching/types.ts`

3. 한글 컬럼명

정산 데이터의 컬럼명이 `처방월`, `거래처명`, `금액` 등 한글입니다. 제약업계 엑셀 양식을 그대로 반영한 것으로, DB 컬럼도 한글입니다. 처음 보면 낯설 수 있지만, 업계 실무자가 엑셀과 1:1로 대조할 수 있도록 의도된 설계입니다.

> 참조: `src/domain/settlement/types.ts` (47개 컬럼 정의)

코드 레이어 구조

DDD(Domain-Driven Design) 원칙을 따라 4개 레이어로 분리되어 있습니다.

디렉토리 맵

```
src/
├── domain/              ← 순수 타입 + 비즈니스 규칙 (프레임워크 의존 없음)
│   ├── user/            UserRepository 인터페이스, User 타입
│   ├── settlement/      SettlementRepository 인터페이스, Settlement 타입
│   ├── cso-matching/    CSOMatchingRepository 인터페이스
│   ├── column-setting/  ColumnSettingRepository 인터페이스
│   ├── email/           EmailLog 타입, EmailService 인터페이스
│   ├── company/         CompanyInfo 타입, EmailProvider 타입
│   └── password-reset-token/
│
├── application/         ← 유스케이스 (도메인 조합, 흐름 제어)
│   ├── user/            RegisterUserUseCase, ApproveUserUseCase
│   ├── settlement/      UploadSettlements, GetSettlements, Export, MonthlySummary
│   └── cso-matching/    UpsertMatchingUseCase
│
├── infrastructure/      ← 외부 서비스 구현체
│   ├── supabase/        Repository 구현체 (Supabase 클라이언트 사용)
│   ├── email/           (제거됨 — lib/email.ts에서 듀얼 프로바이더 직접 처리)
│   └── excel/           ExcelParser (엑셀 → 도메인 객체 변환)
│
└── app/                 ← Next.js App Router (프레젠테이션)
    ├── (auth)/          로그인, 회원가입 등 인증 페이지
    ├── (main)/          메인 레이아웃 (대시보드, 정산 등)
    └── api/             API 라우트 핸들러
```

의존성 방향

```
app/ (프레젠테이션)
  ↓ 호출
application/ (유스케이스)
  ↓ 호출
domain/ (인터페이스 정의) ← infrastructure/ (구현체)
```

핵심 규칙: **domain은 infrastructure를 직접 참조하지 않습니다.** Repository 인터페이스는 `domain/`에 정의하고, 구현체는 `infrastructure/supabase/`에 둡니다. 이렇게 하면 Supabase를 다른 DB로 교체해도 도메인 코드는 변경하지 않아도 됩니다.

"어디를 고쳐야 하는가" 의사결정 표

변경 사항	건드릴 레이어
새 비즈니스 규칙 추가	`domain/` (타입/검증) → `application/` (유스케이스)
새 API 엔드포인트	`app/api/` → 필요시 `application/`

변경 사항	건드릴 레이어
DB 쿼리 수정/최적화	`infrastructure/supabase/`
UI 변경	`app/` 또는 `components/`
새 외부 서비스 연동	`infrastructure/` (새 서비스) → `domain/` (인터페이스)
엑셀 파싱 로직 변경	`infrastructure/excel/`
이메일 템플릿 변경	`lib/email.ts` (듀얼 프로바이더: Resend + SMTP)

인증 흐름

인증은 Supabase Auth를 쓰지 않고 **자체 JWT**를 사용합니다.

미들웨어 (요청 가로채기)

`src/middleware.ts`에서 모든 요청의 쿠키를 확인합니다. 쿠키가 없으면 로그인 페이지로 리다이렉트합니다. 단, 공개 라우트(`/login`, `/register`, `/forgot-password` 등)는 예외입니다.

중요: 미들웨어는 쿠키 **존재 여부**만 확인합니다. JWT 서명 검증은 서버사이드(API 라우트)에서 수행합니다.

서버 (JWT 검증)

`src/lib/auth.ts`의 `getSession()`이 핵심입니다:

- 쿠키에서 JWT 토큰 추출
- `jose` 라이브러리로 HS256 서명 검증
- 만료 확인 (24시간)
- 유효하면 `UserSession` 객체 반환

클라이언트 (상태 관리)

`src/contexts/AuthContext.tsx`가 로그인 상태를 관리합니다:

- `localStorage`에 사용자 정보 저장 (`cso_auth_user` 키)
- React Context로 전역 제공
- SSR 안전성: 마운트 전까지 `null` (hydration 오류 방지)

권한 3요소

모든 권한 판단은 이 세 필드로 이루어집니다:

필드	의미
`is_admin`	관리자 여부 (정산 업로드, 회원 관리 등)
`is_approved`	관리자 승인 완료 여부 (미승인이면 로그인 불가)
`must_change_password`	첫 로그인 시 비밀번호 변경 강제

사용자 수명주기

```
회원가입 요청
  → is_approved = false (승인 대기)
  → 관리자가 승인 (is_approved = true)
  → 첫 로그인 시 비밀번호 변경 강제 (must_change_password = true)
  → 비밀번호 변경 완료 (must_change_password = false)
  → 정상 사용
```

관리자가 비밀번호를 초기화하면 `must_change_password`가 다시 `true`가 되어, 해당 회원은 다음 로그인 시 비밀번호를 변경해야 합니다.

데이터 접근 패턴

모든 라우트는 `infrastructure/supabase/`의 Repository Factory 또는 `application/` Use Case를 직접 사용합니다.

```
// Repository Factory 직접 사용 (단순 CRUD)
import { getUserRepository } from '@/infrastructure/supabase'
const userRepo = getUserRepository()
const user = await userRepo.findByBusinessNumber('1234567890')

// Application Use Case 사용 (비즈니스 로직)
import { authenticateUser } from '@/application/auth'
const result = await authenticateUser(businessNumber, password)
```

Chapter 6

API & 데이터베이스 레퍼런스

개발 중 수시로 찾아보는 참조 문서입니다.

API 공통 규칙

응답 형식: 모든 API는 동일한 형태로 응답합니다.

```
{ "success": true, "data": { ... } }
{ "success": false, "error": "에러 메시지" }
```

인증: 대부분의 API는 `cso_session` 쿠키에 JWT가 있어야 합니다. 서버에서 `getSession()`으로 JWT를 검증하고, 실패 시 401을 반환합니다.

인증 레벨 범례:

- `공개` — 인증 불필요
- `회원` — 로그인 필요 (승인된 회원)
- `관리자` — `is_admin = true` 필요

API 엔드포인트

인증

Method	Path	설명	인증
POST	`/api/auth/login`	로그인 (JWT 쿠키 발급)	공개
POST	`/api/auth/register`	회원가입 (승인 대기 상태로 생성)	공개
POST	`/api/auth/logout`	로그아웃 (쿠키 삭제)	회원
GET	`/api/auth/session`	현재 세션 정보 조회	회원
POST	`/api/auth/forgot-password`	비밀번호 재설정 메일 발송	공개
POST	`/api/auth/reset-password`	토큰으로 비밀번호 재설정	공개
GET/POST	`/api/auth/reset-password-verify`	재설정 토큰 유효성 검증	공개
POST	`/api/auth/change-password`	로그인 상태에서 비밀번호 변경	회원

정산

Method	Path	설명	인증
GET	`/api/settlements`	정산 데이터 조회 (필터/페이징)	회원
GET	`/api/settlements/export`	정산 데이터 엑셀 다운로드	회원
DELETE	`/api/settlements/month/[month]`	특정 월 정산 데이터 삭제	관리자
GET	`/api/settlements/stats`	정산 통계 (건수, 합계 등)	회원
GET	`/api/settlements/monthly-summary`	월별 합계 요약	회원
GET	`/api/settlements/year-months`	조회 가능한 정산월 목록	회원

업로드

Method	Path	설명	인증
POST	`/api/upload/preview`	엑셀 파싱 미리보기 (저장 안 함)	관리자
POST	`/api/upload`	엑셀 파싱 후 DB 저장 확정	관리자

사용자 관리

Method	Path	설명	인증
GET	`/api/users`	전체 사용자 목록	관리자
GET	`/api/users/[businessNumber]`	특정 사용자 조회	관리자
PUT	`/api/users/[businessNumber]`	사용자 정보 수정	관리자
DELETE	`/api/users/[businessNumber]`	사용자 삭제	관리자
POST	`/api/users/approve`	단일 사용자 승인	관리자
POST	`/api/users/approve-batch`	다수 사용자 일괄 승인	관리자
POST	`/api/users/reject`	사용자 가입 거부	관리자
POST	`/api/users/reset-password`	사용자 비밀번호 초기화	관리자
GET/PUT	`/api/users/profile`	내 프로필 조회/수정	회원

이메일

Method	Path	설명	인증
GET	`/api/email/logs`	이메일 발송 이력 조회	관리자
POST	`/api/email/mailmerge`	정산 안내 메일 일괄 발송	관리자

관리자 (CSO 매칭)

Method	Path	설명	인증
POST	`/api/admin/cso-matching/upsert`	CSO 매칭 일괄 등록/수정	관리자
POST	`/api/admin/cso-matching/integrity`	매칭 무결성 검사	관리자

기타

Method	Path	설명	인증
GET/PUT	`/api/columns`	정산 테이블 컬럼 표시 설정	관리자
GET/PUT	`/api/settings/company`	회사 기본 정보 설정	관리자
GET	`/api/system/status`	시스템 상태 확인	회원

데이터베이스 스키마

테이블 관계

```
users (사용자)
  │
  ├── 1:N ── settlements (정산 데이터)     ← business_number로 연결
  │
  └── 1:N ── cso_matching (CSO 매칭)       ← business_number로 연결
                │
                └── settlements.거래처명과 cso_matching.cso_company_name으로
                    업체명 ↔ 사업자번호 매핑

column_settings (컬럼 표시 설정)           ← 독립 (전역 설정)
company_info (회사 정보)                   ← 독립 (전역 설정)
email_logs (이메일 발송 이력)              ← 독립 (감사 로그)
password_reset_tokens (비밀번호 재설정)    ← users.email과 연결
```

핵심 테이블: users

사용자 계정 및 권한 정보.

주요 컬럼	설명
`id`	UUID (PK)
`business_number`	사업자등록번호 10자리 (UK, 사실상의 식별자)
`company_name`	업체명
`email`	대표 이메일
`password_hash`	bcrypt 해시 (salt 12)

주요 컬럼	설명
`is_admin`	관리자 여부
`is_approved`	승인 여부
`must_change_password`	비밀번호 변경 강제 여부

> 타입 정의: `src/domain/user/types.ts`

핵심 테이블: settlements

정산 데이터. SIT솔루션 Compare&Chart에서 다운로드한 엑셀 원본 데이터.

주요 컬럼	설명
`id`	자동 증가 정수 (PK)
`business_number`	CSO 매칭을 통해 결정된 사업자번호
`처방월`, `정산월`	정산 기간
`거래처명`	엑셀 원본 업체명 (CSO 매칭의 키)
`금액`, `수량`, `단가`	정산 금액 정보
`제약_수수료`, `담당_수수료`	수수료
`upload_year_month`	업로드 시점의 연월

컬럼이 47개로, 대부분 한글입니다. 전체 목록은 타입 정의 파일을 참조하세요.

> 타입 정의: `src/domain/settlement/types.ts`

핵심 테이블: cso_matching

엑셀 속 업체명과 회원 사업자번호를 연결하는 매핑 테이블.

컬럼	설명
`cso_company_name`	엑셀에 적힌 업체명 (PK의 일부)
`business_number`	매핑된 사업자등록번호
`created_at`	생성일
`updated_at`	수정일

> 타입 정의: `src/domain/cso-matching/types.ts`

보조 테이블

테이블	설명	타입 정의
`column_settings`	정산 테이블 컬럼의 표시 순서/가시성 설정	`src/domain/column-setting/types.ts`
`company_info`	포털 운영 회사 기본 정보	`src/domain/company/types.ts`
`email_logs`	이메일 발송 이력 (상태, 에러 메시지 등)	`src/domain/email/types.ts`
`password_reset_tokens`	비밀번호 재설정 토큰 (만료 시간 포함)	`src/domain/password-reset-token/types.ts`

Chapter 7

API Reference (외부/내부)

외부 API

국세청 사업자등록정보 진위확인 및 상태조회 서비스

항목	값
출처	공공데이터포털
Base URL	`https://api.odcloud.kr/api/nts-businessman/v1`
프로토콜	REST (POST only)
데이터 포맷	JSON / XML
일 호출 제한	1,000,000건
1회 최대 요청	100건 (초과 시 `TOO_LARGE_REQUEST`)
환경변수	`NTS_API_KEY`
Swagger	https://infuser.odcloud.kr/api/stages/28493/api-docs
버전	v1.1.0

엔드포인트

API	Method	Path	용도
진위확인	POST	`/validate`	사업자번호+개업일자+대표자명 등 일치여부 확인
상태조회	POST	`/status`	사업자번호로 운영상태(계속/휴업/폐업), 과세유형 조회

인증

- `serviceKey`를 **URL query string**으로 전달 (body 아님)
- `returnType=JSON` (기본값) 또는 `returnType=XML`

```
POST https://api.odcloud.kr/api/nts-businessman/v1/status?serviceKey={NTS_API_KEY}
Content-Type: application/json
```

상태조회 API (`/status`)

Request Body:

```
{
  "b_no": ["0000000000"]
}
```

Response (200):

```
{
  "status_code": "OK",
  "match_cnt": 1,
  "request_cnt": 1,
  "data": [
    {
      "b_no": "0000000000",
      "b_stt": "계속사업자",
      "b_stt_cd": "01",
      "tax_type": "부가가치세 일반과세자",
      "tax_type_cd": "01",
      "end_dt": "",
      "utcc_yn": "N",
      "tax_type_change_dt": "",
      "invoice_apply_dt": "",
      "rbf_tax_type": "부가가치세 일반과세자",
      "rbf_tax_type_cd": "01"
    }
  ]
}
```

필드	설명
`b_stt`	납세자상태 (계속사업자, 휴업자, 폐업자)
`b_stt_cd`	납세자상태코드 (`01` : 계속, `02` : 휴업, `03` : 폐업)
`tax_type`	과세유형 메시지
`tax_type_cd`	과세유형코드 (`01` : 일반, `02` : 간이, `03` : 면세 등)
`end_dt`	폐업일자 (YYYYMMDD, 폐업자만)
`utcc_yn`	단위과세전환폐업여부 (Y/N)
`tax_type_change_dt`	최근 과세유형 전환일자
`invoice_apply_dt`	세금계산서 적용일자
`rbf_tax_type`	직전 과세유형 메시지
`rbf_tax_type_cd`	직전 과세유형코드

미등록 사업자 응답: `"국세청에 등록되지 않은 사업자등록번호입니다"`

진위확인 API (`/validate`)

Request Body:

```
{
  "businesses": [
    {
      "b_no": "0000000000",
      "start_dt": "20000101",
      "p_nm": "홍길동",
      "p_nm2": "",
      "b_nm": "",
      "corp_no": "",
      "b_sector": "",
      "b_type": "",
      "b_adr": ""
    }
  ]
}
```

필드	필수	설명
`b_no`	O	사업자등록번호 (10자리 숫자, `-` 제거)
`start_dt`	O	개업일자 (YYYYMMDD, `-` 제거)
`p_nm`	O	대표자성명 (외국인: 영문명)
`p_nm2`		대표자성명2 (외국인 사업자만 한글명)
`b_nm`		상호
`corp_no`		법인등록번호 (13자리 숫자, `-` 제거)
`b_sector`		주업태명
`b_type`		주종목명
`b_adr`		사업장주소 (v1.1 추가)

Response (200):

```
{
  "status_code": "OK",
  "request_cnt": 1,
  "valid_cnt": 1,
  "data": [
    {
      "b_no": "0000000000",
      "valid": "01",
      "valid_msg": "",
      "request_param": { ... },
      "status": { ... }
    }
  ]
}
```

`valid` 값	의미
`01`	일치 (일치 시 status 정보도 함께 반환)
`02`	불일치 (`valid_msg` : "확인할 수 없습니다")

에러 코드

HTTP Status	`status_code`	설명
400	`BAD_JSON_REQUEST`	JSON 포맷 오류
411	`REQUEST_DATA_MALFORMED`	필수 파라미터 누락
413	`TOO_LARGE_REQUEST`	100건 초과
500	`INTERNAL_ERROR`	서버 내부 오류

활용 목적

- 회원가입 시 사업자번호 유효성 검증
- 사업자 상태(폐업/휴업) 사전 확인

내부 API

`POST /api/verify-biz` — 사업자번호 국세청 실시간 인증

국세청 상태조회 API(`/status`)를 프록시하는 서버사이드 엔드포인트. 클라이언트에 API 키를 노출하지 않기 위해 사용.

Request Body:

```
{
  "b_no": "000-00-00000"
}
```

- 하이픈 포함/미포함 모두 허용 (서버에서 정규화)

Response (성공):

```
{
  "success": true,
  "data": {
    "b_no": "0000000000",
    "b_stt": "계속사업자",
    "b_stt_cd": "01",
    "tax_type": "부가가치세 일반과세자"
```

```
  },
  "verified_at": "2026. 02. 25. 14:30:22"
}
```

Response (실패):

```
{
  "success": false,
  "error": "에러 메시지",
  "code": "ERROR_CODE"
}
```

code	HTTP	설명
`CONFIG_ERROR`	500	`NTS_API_KEY` 환경변수 미설정
`INVALID_REQUEST`	400	JSON 파싱 실패
`MISSING_BNO`	400	`b_no` 누락
`INVALID_FORMAT`	400	10자리 숫자 아님
`NOT_REGISTERED`	200	국세청 미등록 사업자
`NTS_API_ERROR`	502	국세청 API 비정상 응답
`TIMEOUT`	504	10초 타임아웃 초과
`NETWORK_ERROR`	502	네트워크 오류

사용처: 회원가입 페이지 (`/register`) — 사업자번호 10자리 입력 완료 시 자동 호출

Chapter 8

Changelog

이 프로젝트의 모든 주요 변경사항을 기록합니다. Semantic Versioning 및 Keep a Changelog 형식을 따릅니다.

[0.16.0] - 2026-02-26

Changed

- **DDD 레이어 전환 완료** — `lib/db.ts` 호환 레이어(346줄, 48개 함수) 완전 제거
  - 28개 API 라우트: `@/lib/db` → Repository Factory 또는 Application Use Case 직접 호출로 전환
  - `lib/email.ts` : `@/lib/db` 의존 제거, Repository Factory 직접 호출
  - 10개 테스트 파일: mock 대상을 `@/lib/db` → `@/infrastructure/supabase` 또는 `@/application/auth` 로 전환

Added

- Auth Use Case 3개 신규 생성 (로그인 비즈니스 로직을 라우트에서 분리)
  - `LoginUseCase` — 사용자 인증, 계정 잠금, 실패 카운트, 리다이렉트 결정
  - `ForgotPasswordUseCase` — 비밀번호 재설정 요청 (사용자 조회 + 이메일 매칭 + 토큰 생성)
  - `ResetPasswordVerifyUseCase` — 토큰 검증 + 비밀번호 변경 완료
- Auth Use Case 테스트 3개 (21개 케이스)
- 전체 테스트: 27개 파일, 171개 케이스

Removed

- `src/lib/db.ts` — 모든 함수가 Repository 1줄 위임하는 호환 레이어 삭제

[0.15.1] - 2026-02-25

Added

- Tier 1~2 테스트 커버리지 확장 — 10개 테스트 파일, 40개 케이스 추가 (총 150개)
  - **Application Use Case**: GetMonthlySummaryUseCase (4개)
  - **API Route**: auth/logout (2), auth/session (3), settlements/export (4), settlements/stats (3), settlements/year-months (3), users (4), users/approve (5), users/reject (5), columns (7)
  - 신규 커버: auth/logout 100%, auth/session 100%, columns 80%, settlements/export 91%, users/approve 86%, users/reject 89%

[0.15.0] - 2026-02-25

Added

- 테스트 인프라 풀세트 구축 — Vitest + Testing Library + Playwright
  - 14개 테스트 파일, 108개 테스트 케이스 전체 통과
  - **Unit (45개)**: `lib/auth` (해싱, JWT, 포맷, 검증), `lib/utils` (cn), `lib/excel` (validateExcelFile, exportToExcel, getYearMonthsFromData)
  - **Application Use Case (30개)**: RegisterUser, ApproveUser, GetSettlements (관리자/일반/매칭없음 분기), UploadSettlements, ExportSettlements, UpsertMatching
  - **API 통합 (21개)**: login (400/401/403/200 + 리다이렉트 3분기), register (밸리데이션 + 중복 체크), settlements (인증/권한/검색/페이지네이션)
  - **컴포넌트 (12개)**: AuthContext (세션 관리, localStorage 동기화), Header (로딩/관리자/일반 메뉴)
  - **E2E (7개)**: 로그인/로그아웃 플로우, 정산서 접근 (Playwright)
- 테스트 헬퍼: 7개 Repository mock 팩토리, 테스트 픽스처, NextRequest 생성 헬퍼
- npm scripts 8개 추가 (`test`, `test:watch`, `test:unit`, `test:integration`, `test:component`, `test:coverage`, `test:ui`, `test:e2e`)
- 설정 파일: `vitest.config.ts`, `playwright.config.ts`, `src/__tests__/setup.ts`

[0.14.1] - 2026-02-25

Changed

- 정산서 Notice 편집 UI를 사이트 설정(`/admin/settings`)에서 마스터조회(`/admin/master`)로 이동
  - Notice amber 카드에 인라인 편집 버튼 + Dialog 추가 (WYSIWYG 편집)
  - 사이트 설정 페이지에서 Notice 카드 제거 — 브랜딩 + 이메일 인프라만 유지
  - 변수 도움말(`{{정산월}}`, `{{정산월+1}}`, `{{대표자명}}`) 및 기본값 초기화 기능 포함
  - 일반 사용자 대시보드(`/dashboard`) Notice 읽기 전용 표시는 변경 없음

[0.14.0] - 2026-02-25

Added

- 이메일 듀얼 프로바이더 (Resend API + SMTP 하이웍스)
- 메일머지 SSE 실시간 진행률 표시
- 대시보드 시스템 정보 강화 (SMTP/NTS 상태, 버전 동기화)

Fixed

- 라인 엔딩 정규화 (CRLF → LF)

[0.13.0] - 2026-02-25

Added

- 회원가입 사업자번호 국세청 실시간 인증 통합

[0.12.0] - 2026-02-24

Changed

- DDD 구조 개편 Phase 1 — `domain/` `infrastructure/` `application/` 레이어 분리
- `docs(h)/` → `docs/` 폴더 통합

Added

- 팀 온보딩용 문서 5종 (ARCHITECTURE, API_REFERENCE, DATABASE, DEPLOYMENT, TROUBLESHOOTING)

[0.3.0] - 2026-02-25

Added

- `profile_complete` 플래그 분리 — 비밀번호 변경과 프로필 완성 독립 관리

[0.2.0] - 2026-02-25

Added

- DB 기반 사용자 계정 일괄 생성 + 첫 로그인 이메일 입력 강제
- 첫 로그인 시 전체 회원정보 입력 강제 + 초기 비밀번호를 사업자번호 전체로 변경

Fixed

- 프로필 미완성 판단을 DB 직접 조회로 변경

Pre-release (2026-01-28 ~ 2026-02-19)

버전 태그 도입 전 개발 이력. 초기 시스템 구축 및 핵심 기능 구현.

인프라 & 배포 (2026-01-28 ~ 2026-02-19)

- Supabase 데이터베이스 연결 및 클라이언트 초기화

- Netlify 배포 설정 + Functions 리전 도쿄 설정
- Vercel Speed Insights 추가
- Auth RLS 초기화 플랜 (4 테이블)
- `cso_matching` Service role RLS 정책 추가
- 회사 정보 API on-demand revalidation 적용
- `cookies()` 사용 API 라우트에 `force-dynamic` 일괄 추가
- CLAUDE.md 추가 (공통 규칙 @import)

정산서 조회 (2026-01-28 ~ 2026-02-13)

- CSO 정산서 웹 조회 시스템 초기 구현
- 관리자용 정산서 마스터 조회 페이지
- 정산서 조회 피벗 형태 (거래처명별 소계 + CSO관리업체 총합계)
- 정산서 Notice 영역 및 수수료 합계 문구 추가
- Notice 변수 치환 (`{{정산월}}` , `{{정산월+1}}` , `{{대표자명}}`)
- 정산 API SELECT 최적화 — 필요한 컬럼만 조회
- 월별 수수료 합계 UI 개선 및 검색 최적화
- Supabase 1000건 조회 제한 해결 (페이지네이션)
- 정산서 조회 무한 로딩 해결 — 모든 에러 케이스 처리
- 데이터 없음 안내 메시지 개선

무결성 검증 (2026-02-04 ~ 2026-02-11)

- 정산서 무결성 검증(CSO 매칭) 기능 추가
- 관리자 직접 매칭 등록/수정/삭제 UI
- 통계 카드 클릭 필터링 + 검색 개선 + 거래처 관리 UI
- 키보드 중심 인라인 편집 + 낙관적 업데이트 + 에러 우선 정렬
- 필터 카드 집계 기준 수정 및 전체 UI 개선

회원 관리 (2026-01-29 ~ 2026-01-31)

- 프로필 페이지 새 필드 추가 및 다음 주소 검색 적용
- 회원관리 엑셀 다운로드 및 전화번호 포맷 개선
- 주소 저장 구조 변경 (우편번호, 주소1, 주소2 분리)
- 회원 승인 페이지에 일괄 승인 기능 추가
- 비밀번호 재설정/변경 강제 기능 구현
- Resend API 기반 이메일 비밀번호 재설정

이메일 & 알림 (2026-01-28 ~ 2026-01-30)

- 업로드 후 이메일 발송 선택 다이얼로그
- 모든 이메일 템플릿에 회사 정보 푸터 적용

- 메일머지 변수 재연동 및 사용 가능 변수 확장
- 엑셀 업로드 컬럼 매핑 기능 추가
- 월별 수수료 합계 조회 기능 구현

사이트 설정 & UI (2026-01-28 ~ 2026-02-09)

- 로그인 화면 푸터 회사정보 설정 기능
- 저작권/라이선스 표시 추가 및 간소화
- AuthContext 기반 전역 인증 상태 관리
- 데이터 관리 페이지 — 정산월별 상세 정보 표시
- Supabase 병원정보 자동 동기화 시스템 구축
- 병원 마스터 데이터 관리 스크립트 추가
- CSO매칭 기반 정산서 조회 API 권한 제어